

Design and Evaluation of a Deterministic Query Orchestrator for a Distributed Hybrid Retrieval System

Msc Computer Science



**UNIVERSITY
OF LONDON**

Software Project Report
Devang Chauhan

10402 Words

Excluding title page, abstract, table of content, tables, references and appendix

Abstract

Placeholder

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	6
List of Tables.....	6
1. Introduction.....	7
1.1 Problem Statement.....	7
1.2 Contribution	8
1.3 Aim and Objectives.....	8
2. Literature Review	10
2.1 RAG.....	10
2.2 NL2SQL.....	10
2.3 Resource Selection	11
2.4 Vector Search.....	11
2.5 Relational Search	12
2.6 Distributed Systems	12
2.7 Query Planning.....	12
2.8 Synthesis and Research Gap.....	12
3. Methodology	14
3.1 Scope	14
3.2 Database	15
3.2.1 Selection and Preparation	16
3.2.2 Layout and Sharding	17
3.2.4 Embeddings	20

3.2.5 Indexes	21
3.3 Software Architecture	22
3.3.1 Components and Technology Choices	22
3.3.2 Relational Orchestration	25
3.3.3 Vector Retrieval and Enrichment	25
3.3.4 Validation, Error Handling and Timing	28
3.3.5 Startup and Deployment	29
3.4 Verification and Testing	29
3.5 Evaluation Method	30
3.5.1 Benchmark Queries and Ground Truth	30
3.5.2 Correctness, Latency and Retrieval Effectiveness	31
3.5.3 Benchmark Execution	32
3.5.4 Strategy Comparisons and Report Artifacts	33
3.6 Reproducibility	33
3.7 Data Use and Credentials	34
4. Results and Analysis	35
4.1 Benchmark Outcomes	35
4.2 Verification Findings	37
4.3 Relational Retrieval and Shard Routing	38
4.4 Filter Order and Vector Latency	38
4.5 Retrieval Effectiveness and Result Completeness	40
4.6 Layout, Routing and Timing Boundaries	42
5. Evaluation	44
5.1 Achievement of the Objectives	44
5.2 Interpretation in Relation to the Literature Review	44
5.3 Validity and Generalisability	45

5.3.1 Workload and Relevance Assessment	45
5.3.2 Experimental Control and Statistical Interpretation	46
5.3.3 Deployment and Configuration Boundaries.....	46
5.4 Engineering Strengths and Remaining Limitations	47
5.5 Further Work.....	47
6. Conclusion.....	49

List of Figures

Figure 1: Project scope	15
Figure 2: Logical shards	16
Figure 3: Industry classification	17
Figure 4: Relational database schema	19
Figure 5: Vector database schema	20
Figure 6: Deployment	23
Figure 7: Orchestrator	24
Figure 8: Filter-then-search	26
Figure 9: Search-then-filter	27
Figure 10: Mean orchestrator latency by query and configuration	36
Figure 11: Matched-query orchestrator latency differences	40
Figure 12: Strategy-group retrieval quality	41

List of Tables

Table 1: Overall outcomes	35
Table 2: Matched relational comparisons	38
Table 3: Matched execution order comparisons	39
Table 4: Result counts by execution order	42
Table 5: Assessment of project objectives	44

1. Introduction

This report begins by establishing the way this project aims to contribute to addressing the challenges involved in the enterprise retrieval systems. It reviews the related work and then moves on to explain the methodology adopted in the design of the orchestrator, followed by the results recorded in the use of the orchestrator against a gold standard benchmark, and lastly, the evaluation and discussion of those results.

1.1 Problem Statement

The advent of large language models (LLMs) [1] has opened up numerous applications where a conversation can be had with an LLM using the knowledge encoded in the parameters of the model during pre-training. While fascinating, this has a limitation when it comes to their use within enterprises. Enterprise data is often gated and hence is not included in the pre-training of the LLMs. This can lead to hallucinations where the LLM confidently responds to queries with false information where it does not have innate knowledge. Retrieval augmented generation (RAG) [2], while based on techniques long known in the domains of search and recommendation systems, has been found to be a reliable way to ground the responses of LLMs in relevant and recent information to reduce hallucinations [3].

RAG is extensively used to provide gated enterprise knowledge to LLMs to ground their responses. Depending on the shape of information, the enterprise knowledge is often distributed across several knowledge stores such as vector stores, document stores, relational tables and graph databases. As this knowledge grows, the knowledge retrieval systems start taking the form of a distributed system where these knowledge stores need to be either partitioned or replicated or both depending on whether consistency or availability is targeted [4].

Also, in enterprise environments, deterministic orchestration is often preferred because retrieval decisions are predictable and auditable, allowing system designers to understand why particular data sources were queried for a given request. However, the performance implications of adopting deterministic orchestration strategies in distributed hybrid retrieval systems remain insufficiently understood.

Little work evaluates how different deterministic orchestration strategies influence end-to-end retrieval latency and retrieval quality when semantic and structured retrieval must be coordinated across distributed knowledge stores. Consequently, practitioners have limited evidence regarding which orchestration strategies provide

the best trade-off between latency and retrieval quality under different sharding configurations.

Lastly, in any information retrieval system, it is important to maintain a balance between the speed and accuracy of the response served to the user to maintain a higher degree of user delight [5]. This project addresses these concerns through its specific contribution.

1.2 Contribution

This project contributes the design and implementation of a deterministic query orchestrator and an experimental comparison of routing and execution strategies within a common sharded retrieval system. It evaluates how these strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations. Here, deterministic strategies follow explicitly defined routing and coordination rules for a given input and system state.

In the information-retrieval literature, hybrid retrieval [6] commonly refers to combining sparse lexical and dense semantic retrieval signals into a unified ranking. The system developed in this project does not perform score or rank fusion. Instead, it is a query orchestrator for a hybrid retrieval system (HRS) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data through unified orchestration. Here, aligned data refers to corresponding information represented across the different retrieval stores.

1.3 Aim and Objectives

The aim of this project is to design and implement a deterministic query orchestrator for a distributed HRS, and experimentally evaluate how routing and execution strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations.

To achieve this aim, the project has the following objectives:

1. Design and implement a query orchestration framework exposed through a stateless hypertext transfer protocol (HTTP) application programming interface (API) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data.
2. Implement and compare deterministic shard-routing strategies, including broadcast and selective shard querying, and execution strategies, including filter-then-search

and search-then-filter, for applicable query types.

3. Establish a reproducible evaluation benchmark comprising development and held-out test queries, with documented ground-truth results and relevance judgements appropriate to each query type.

4. Evaluate the strategies under controlled workloads and sharding configurations using mean, P95 and P99 retrieval latency [7], precision, recall and F1-score [5], and NDCG@k for ranked results [8]. For known-item queries, measure success as the proportion of queries for which the designated target appears within the top k results.

5. Analyse the trade-offs between latency and retrieval effectiveness, identifying the workload and configuration conditions under which each strategy offers advantages, and explaining the limitations of the findings.

The sections following literary review justify the methods selected to address these objectives, describe their implementation, and evaluate the extent to which the objectives were achieved.

2. Literature Review

This project lies at the intersection of RAG, natural language to SQL (NL2SQL) [9], federated search, vector search, relational databases, and distributed systems. Consequently, the literature review draws upon contributions from each of these domains to motivate the proposed architecture and identify the research gap.

2.1 RAG

RAG has received considerable attention from the research community. Xu et al. [10] propose a distributed RAG (DRAG) system using a peer-to-peer (P2P) network of knowledge stores and local LLMs. Their healthcare-focused approach introduces a novel algorithm that retrieves information locally before querying neighbouring peers. While the system is distributed, it relies solely on vector embeddings [3] as grounding information for LLMs and therefore does not address an HRS, where structured data must also be queried. Similarly, Guerraoui et al. [11] acknowledge the distributed nature of modern retrieval systems and propose RAGROUTE, a neural network classifier that predicts which knowledge store should be queried. Although this reduces unnecessary communication, the routing model is probabilistic and requires retraining whenever the underlying knowledge stores are updated. Recognising that enterprise retrieval extends beyond embeddings, Yu et al. [12] propose TableRAG, which embeds relational table schemas alongside textual content so that retrieved schema embeddings can guide subsequent SQL queries. While this demonstrates the value of hybrid retrieval, the paper does not discuss schema evolution common in large distributed tables are handled in practice.

2.2 NL2SQL

As structured data becomes an increasingly important component of HRS, the quality of NL2SQL translation has a direct impact on retrieval quality. Existing approaches can broadly be categorised as rule-based, neural and hybrid methods. Shalaan et al. [13] propose Guided SQL (GSQL), which combines rule-based processing, natural language processing and vector embeddings to improve SQL generation. SQLizer adopts a different approach by first constructing a query sketch before iteratively materialising it against the database schema using CoreNLP and Word2Vec [14]. More recently, Liu et al. [9] observe that although rule-based systems provide explainability, they struggle with adaptability and complex schemas, while neural approaches remain constrained by training data. Consequently, the field has shifted towards LLM-based methods and fine-tuning techniques that demonstrate stronger generalisation across complex database schemas [9]. Given this direction,

implementing a custom NL2SQL engine would contribute relatively little to the primary aim of this project, which is the design and evaluation of query orchestration rather than query translation itself.

2.3 Resource Selection

Once queries have been translated, the retrieval system must determine which resources should be searched. Federated search addresses this problem by providing a unified interface through which multiple independent collections can be searched and their results combined [15]. Garba et al. [15] identify CORI [16] and ReDDE [17] as two of the most influential resource selection techniques. Callan et al. [16] use probabilistic inference networks to estimate the likelihood that a collection contains relevant information. Here, CORI relies primarily on term frequency (TF) and inverse document frequency (IDF), limiting its ability to capture semantic similarity. Si et al. [17] improve resource selection by estimating collection sizes and emphasise that reducing communication between distributed collections becomes increasingly important as the number of knowledge stores grows. Although these approaches were developed for keyword-based search rather than an HRS, these studies establish that resource selection remains a fundamental challenge in distributed retrieval.

2.4 Vector Search

Within a selected resource, semantic retrieval identifies relevant items by representing documents and queries as vector embeddings, allowing similarity to be measured using functions such as cosine similarity or Euclidean distance rather than exact keyword matching [4]. However, efficiently searching large vector collections presents another challenge. Flat indexes provide exact retrieval but exhibit linear search complexity, which can become costly as dataset size and query volume increase [4]. Modern vector databases therefore employ approximate nearest neighbour (ANN) indexes such as inverted file index (IVF) and hierarchical navigable small world index (HNSW), which can reduce retrieval latency with index configuration affecting the trade-off between search speed and retrieval quality [4, 18]. More recent approaches, including DiskANN and SPANN, further improve scalability by storing large portions of the index on SSDs while maintaining lightweight routing structures in memory, enabling efficient retrieval over billion-scale datasets [19, 20]. These approaches substantially improve vector search performance but focus on optimising retrieval within individual vector stores rather than across distributed knowledge stores.

2.5 Relational Search

While semantic retrieval addresses unstructured information, enterprise knowledge also resides within relational databases. Relational systems typically employ B-tree indexes for range queries, hash indexes for equality comparisons, while PostgreSQL also provides GIN indexes for full-text and JSON search [4, 21]. Kleppmann and Riccomini [4] note that although indexes improve retrieval performance, they increase storage requirements and slow write operations because they must be maintained as the underlying data changes. Consequently, index selection represents a trade-off between query performance and maintenance overhead.

2.6 Distributed Systems

As enterprise knowledge continues to grow, a single machine may no longer be sufficient to store or process all data. Distributed database systems therefore introduce concerns such as sharding and replication. The degree of fragmentation directly influences query performance [22], while the choice of sharding strategy depends heavily on workload characteristics. Key-range partitioning is effective when queries exhibit locality, whereas hash-based partitioning generally provides a more balanced distribution of reads and writes [4]. Similar principles apply to vector databases, where semantically similar vectors may be clustered together to reduce the search space during retrieval [23]. Distributed systems also introduce opportunities for parallel query execution; however, Özsu and Valduriez [24] note that reducing response time through parallelism may increase overall execution time because of additional communication overhead. Consequently, computation should occur as close to the data as possible whenever feasible [24].

2.7 Query Planning

These considerations ultimately influence query planning within distributed retrieval systems. Özsu and Valduriez [25] argue that query optimisation should minimise communication costs by reducing the amount of data transferred between nodes. They recommend assigning execution costs to alternative query plans and favour strategies that perform selection as early as possible, thereby reducing intermediate results and avoiding expensive join operations wherever possible [25]. Such observations reinforce the importance of orchestration strategies that intelligently coordinate retrieval across multiple knowledge stores.

2.8 Synthesis and Research Gap

The reviewed literature demonstrates progress in semantic retrieval, distributed RAG,

resource selection, relational and vector indexing, and query planning. Together, these contributions show that retrieval performance depends on both the efficiency of individual stores and their coordination across a distributed system. Resource selection motivates querying only relevant collections, distributed systems research highlights communication overhead, and query planning emphasises reducing intermediate results through early selection. These findings motivate examining both which shards are queried and the order in which filtering and retrieval are performed.

However, the reviewed work does not establish how alternative deterministic orchestration strategies compare within a common sharded system supporting structured relational, dense semantic, and sparse lexical retrieval. This project addresses that gap by comparing broadcast and selective shard querying, alongside filter-then-search and search-then-filter execution where applicable, over aligned distributed data. The evaluation examines the trade-offs between retrieval latency and effectiveness across defined workloads and sharding configurations. The following section describes the development methodology and experimental approach used to implement the system and its evaluation.

3. Methodology

A deterministic query orchestrator was developed to address the aim and objectives of the project. The two governing themes during the development were testability and reproducibility. These themes led to the development of the project in the small increments of data acquisition, schema design, shard construction, relational retrieval, vector retrieval, orchestration, end-to-end integration, and benchmarking. All these increments are covered in the following sub-sections.

3.1 Scope

In real world, an HRS receives natural language queries that may require information from multiple knowledge stores. A decision has to be made as to which knowledge stores should be queried, in what order they should be queried, and how the retrieved results should be combined before returning a response. In distributed environments, these decisions directly influence communication overhead, response latency and retrieval quality.

HRS often encounters complex queries. For example:

"Give us three movie options directed by an academy award-winning director that will motivate us to survive through tough times in life."

Such a query needs to be decomposed into the following sub-queries:

Query 1: *"Find movies directed by Academy Award-winning directors."*

Query 2: *"Find movies whose plot summaries describe inspirational stories."*

In real world enterprise applications, LLMs are increasingly used to generate such sub-queries from a complex user query and to synthesise a final response from the retrieved information [26]. These components, however, are not the focus of this project and are therefore kept outside the scope [Figure - 1].

Instead, this project focuses designing the deterministic orchestration layer as an HTTP API responsible for coordinating retrieval across knowledge stores. In a production enterprise application, an LLM will decompose a natural language query into discreet queries the orchestrator can understand, and will request the orchestrator API to retrieve information from the aligned data in knowledge stores.

A user interface, authentication, data update endpoints, and cloud deployment are also kept outside the scope.

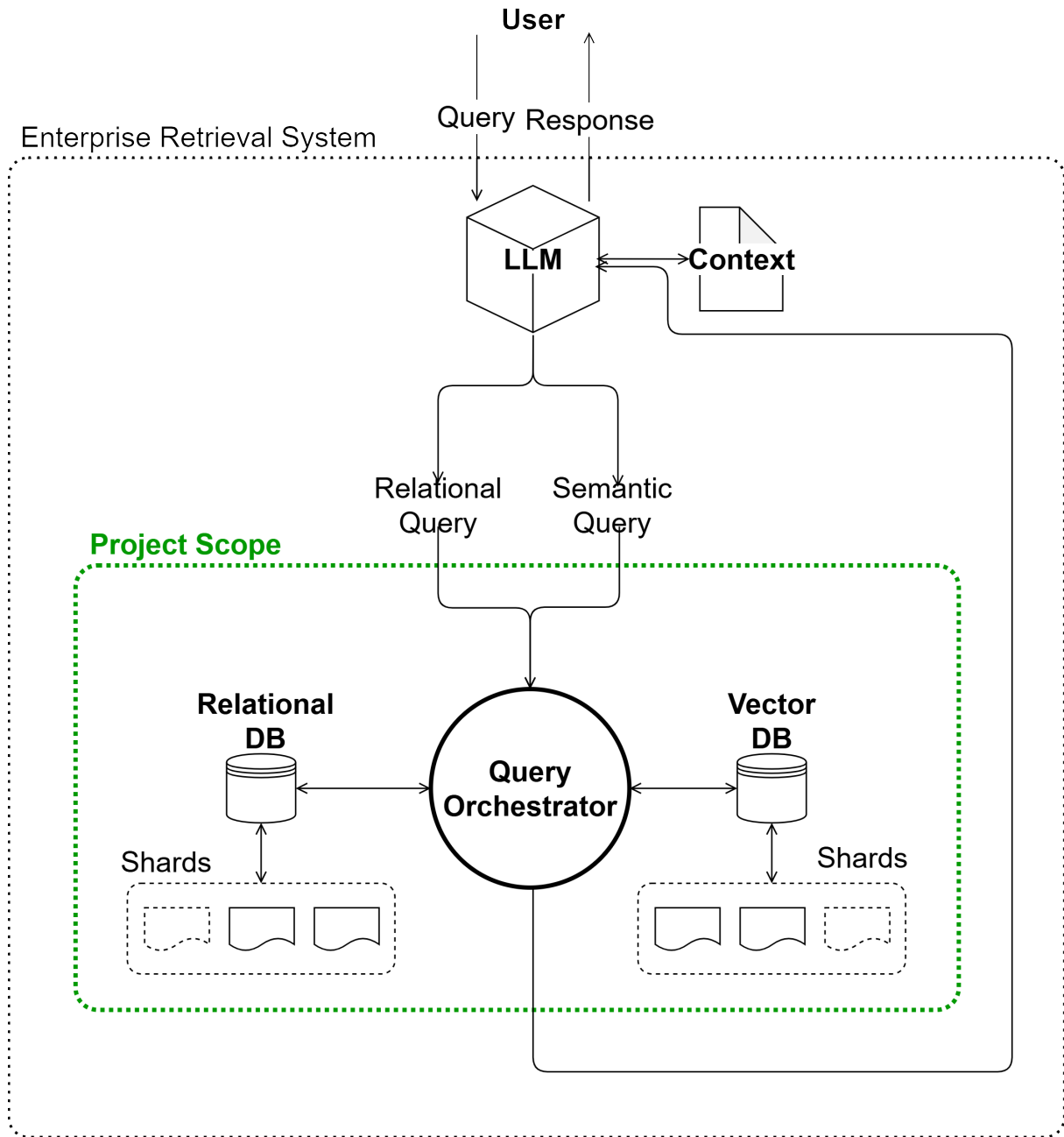


Figure 1: Project scope

3.2 Database

To perform evaluation of the orchestrator, it needs to be able to retrieve information from knowledge stores. Since movies contain structured data such as title, release year, director, and descriptive data such as overview, they were selected to build such knowledge stores. Among other alternatives, the movie database (TMDB) [27] was chosen for its free API access to its data, to build the knowledge stores.

PostgreSQL was selected for the relational store because its SQL support, integrity constraints, and transactions suited the structured movie metadata and relationships,

enabling precise filtering and consistent data ingestion. Qdrant [28] was selected for the vector store because it supported dense and sparse retrieval, named vectors, and metadata filtering.

3.2.1 Selection and Preparation

Using TMDB’s discovery filters, two groups were defined: U.S.-origin English-language films and Indian-origin Hindi-language films. Both applied a primary release-date range from 1 January 1990 to 1 August 2026. These specific languages and the time period were chosen for the author’s familiarity with the English and Hindi movies during the period, which was crucial for creating the known-item queries during evaluation.

Movies with a minimum 20 votes were selected in the descending order of popularity, with the exclusion of adult content. A total of 10,000 records were collected from the larger United States/English result set, while total 1,093 records from the India/Hindi group were collected. There were no duplicate movie identifiers between the groups.

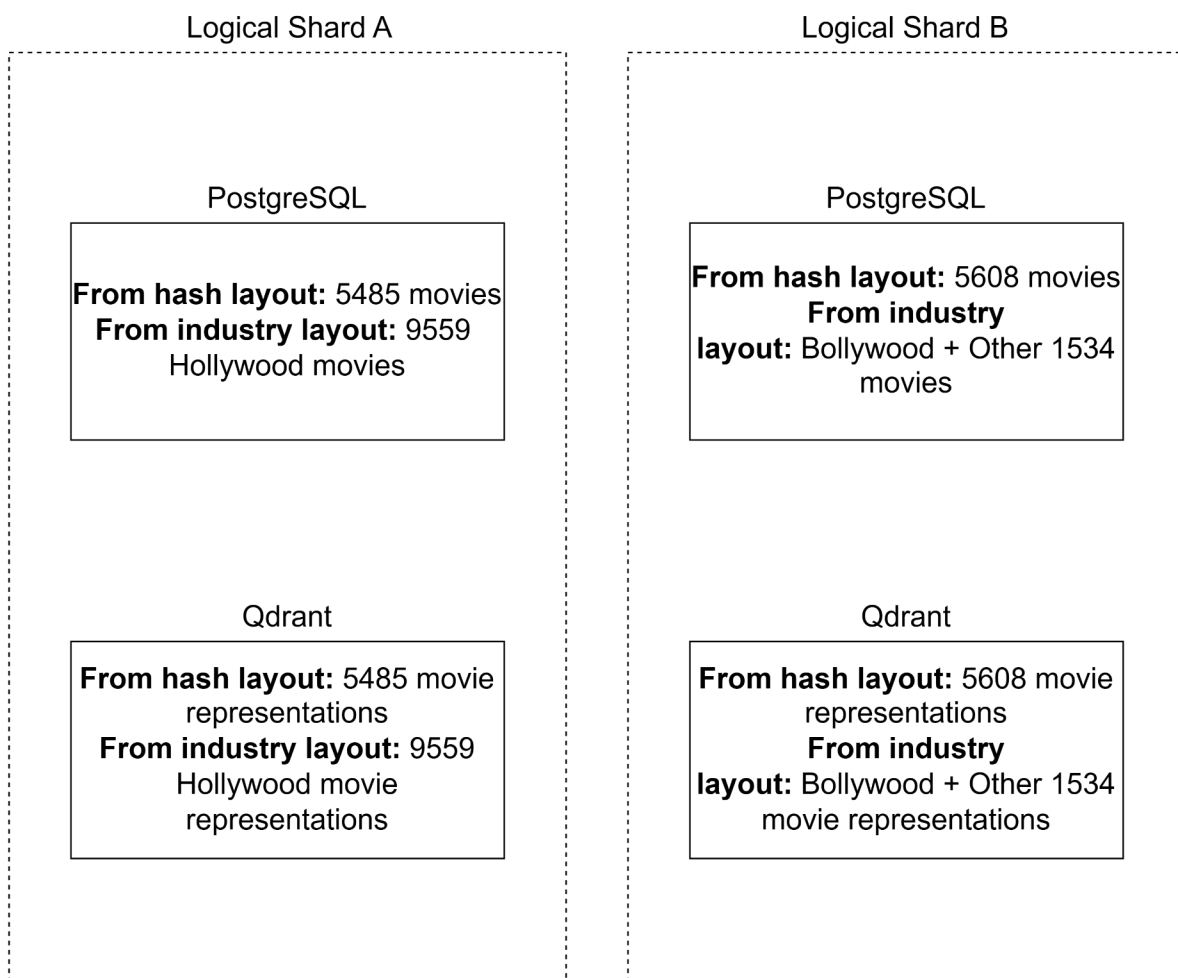


Figure 2: Logical shards

Records required a non-blank title and overview, resulting in a dataset of 11,093 movies.

Raw responses were cached as JSON before normalisation so that data preparation could be repeated without further API requests to TMDb. From the raw response, the canonical dataset was prepared that retained titles, overviews, release dates, genres, directors, cast, production countries, and a derived industry label. The canonical dataset served as the common input to the Python [29] loader for both the databases. SHA-256 [30] checksums [31] were created out of the canonical dataset, which were validated before loading the data so that complete parity was maintained.

3.2.2 Layout and Sharding

Figure 2 shows the two sharding configurations that were developed from the same canonical dataset for the experimental evaluation. The hash layout assigned even movie identifiers to shard A and odd identifiers to shard B, producing 5,485 and 5,608 movies respectively. This provided an approximately balanced distribution, although

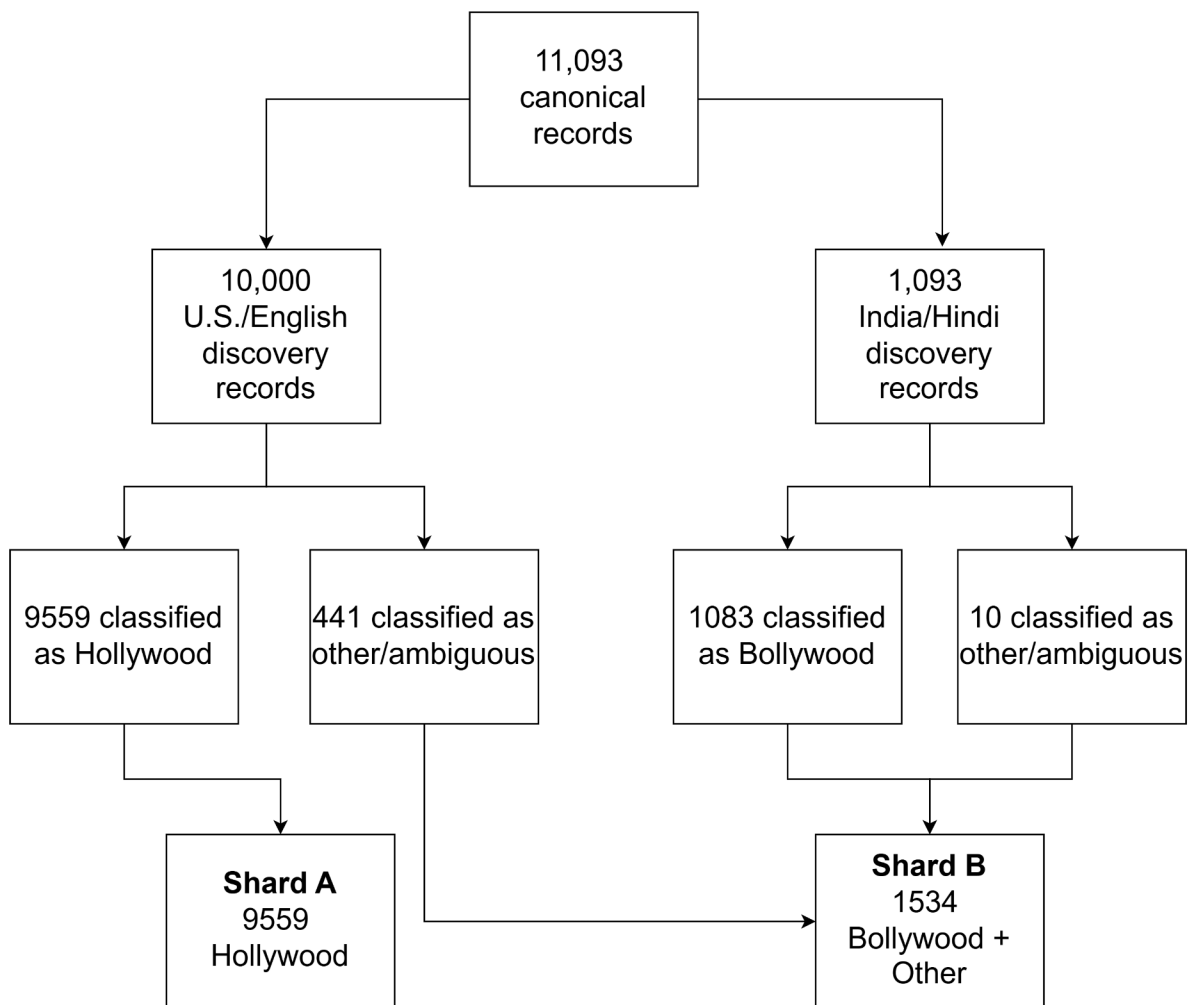


Figure 3: Industry classification

the assignment did not reflect the content of the movies. Figure 3 shows how 11,093 records were split into shards A and B.

Hashing and theme based sharding are one of the most common sharding strategies [4]. Therefore the design adopted both and also evaluated their impact during evaluation. In order to keep the experiment locally reproducible, only two shards were simulated. Simulating a larger dataset distributed across a larger number of shards is a natural future extension of this project.

3.2.3 Schema

Two complementary schemas were developed. Figure 4 shows the PostgreSQL tables and their relationships, while figure 5 shows a Qdrant point, its named vectors and its payload. The schemas represent the same movies in forms suited to exact relational conditions and ranked retrieval respectively.

In PostgreSQL, `movies` holds the attributes belonging directly to a movie, including its title, release date, overview, runtime, votes and industry. Genres, credits and countries occupy separate tables because a movie may have several of each, and each genre, person or country may relate to many movies. The `movie_genres` and `movie_countries` use composite primary keys to prevent duplicate associations. The `movie_credits` table links movies to people and retains information specific to a credit, including actor or director role, character name and cast order. Placing those attributes on the credit preserves the distinction between a person and that person's participation in a particular movie.

Qdrant stores one point per movie within each layout's collection. Each point contains the named vectors `overview_dense`, `full_document_dense` and `metadata_sparse`. Keeping these representations on one point gives them a common identity while allowing their contents to differ. The payload has duplicated fields needed for metadata filtering: `release_year`, `genre_ids`, `industry` and `country_codes`. However, PostgreSQL remains the authoritative metadata store.

The same `movie_id` connects both databases. The canonical TMDb identifier is stored as the primary key `movies.movie_id` in PostgreSQL, as the Qdrant point ID, and as `movie_id` in that point's payload. Thus, for a movie with identifier `m`, the relational record and vector point both retain `m` rather than receiving unrelated store-specific identifiers. This avoids matching by title, which may be shared by different movies or vary in spelling, and removes the need for a separate identifier-mapping table. Association-table foreign keys also use this value to connect the movie to its

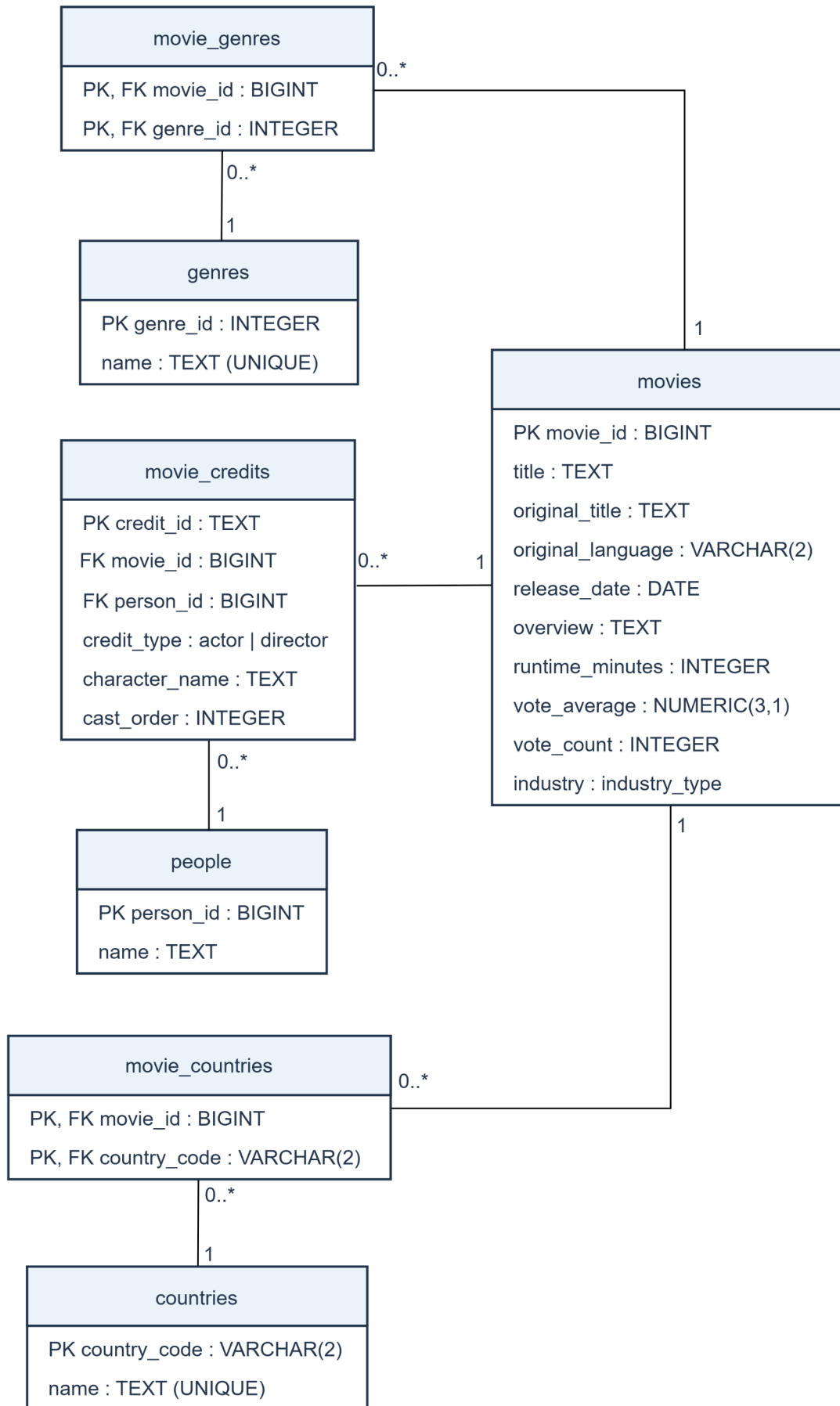


Figure 4: Relational database schema

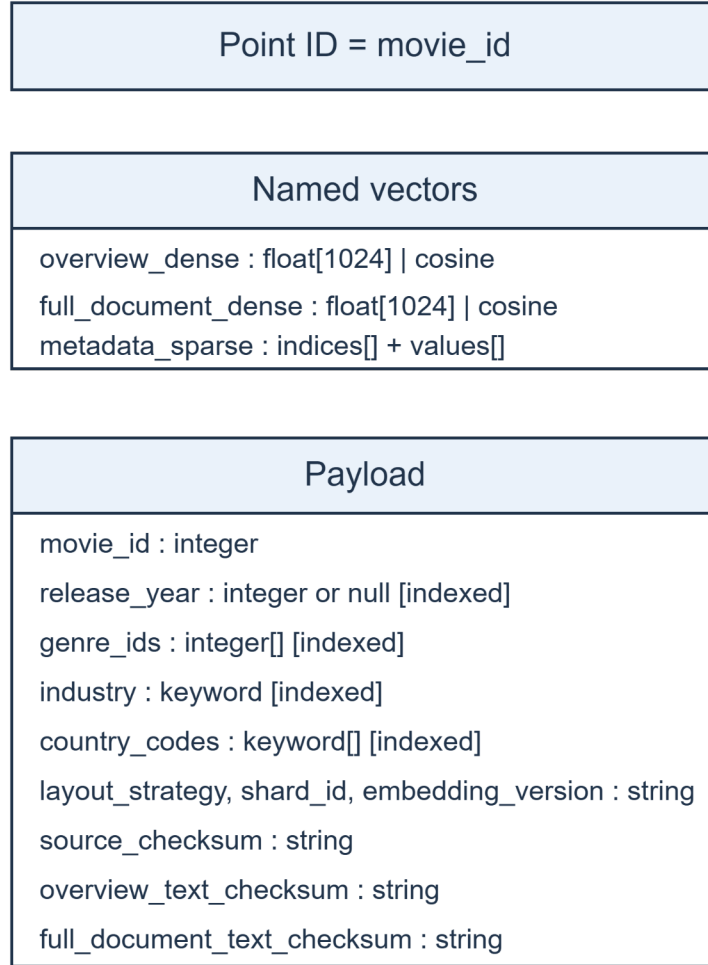


Figure 5: Vector database schema

relational metadata.

3.2.4 Embeddings

overview_dense embedded the overview alone. metadata_sparse encoded eight newline-separated labelled fields in this order: Title, Original title, Release year, Genres, Industry, Countries, Directors, and Cast. List entries were comma-separated; genres, countries and directors retained their canonical source order. Cast contained up to the first ten billed performers, formatted as “name as character”, or name alone when the character was absent. full_document_dense used the same metadata template followed by an “Overview:” line. White space within values was collapsed, preserving case and punctuation. Missing or blank values became “[unknown]”, and empty lists became “[none]”. Beyond the ten-person cast limit, the loader applied no character or token length truncation before passing text to the embedding libraries.

Dense embeddings were generated using text-embedding-3-large [32] with 1,024 dimensions and compared using cosine distance [33, 34]. Sparse representations were generated using Qdrant/bm25 [35] through FastEmbed [36] as term indices and

weights. The dense representations supplied semantic evidence from text, while the sparse representation retained lexical evidence from metadata. These representations were stored under the named-vector fields described in figure 4.

The 1,024-dimensional setting and selected model were not independently optimised in this project and should not be interpreted as demonstrated best choices. Dense movie embeddings were generated once, retained and loaded into both layouts, avoiding repeated external generation and ensuring that a layout comparison used the same vectors.

3.2.5 Indexes

Relational Indexes

PostgreSQL used B-tree [37] indexes on `movies.release_date`, `movies.industry`, `movies.title` and `people.name` to support date-range and equality conditions.

Relationship indexes on `movie_genres.genre_id` and `movie_countries.country_code` supported finding movies associated with a particular genre or production country. An index on `movie_credits.movie_id` supported retrieving a movie's credits, while the composite index (`person_id`, `credit_type`) supported finding movies associated with a person in a specified role, such as director.

Vector Indexes

Qdrant 1.13.6 was deployed without overrides to its default HNSW settings for dense indexing. Sparse representations used an inverted index.. The dense configuration supported similarity search over the 1,024-dimensional embeddings, while the sparse index supported retrieval using term indices and weights. Qdrant's inverse document frequency modifier was enabled for sparse retrieval, so term weighting depended on the contents of each collection.

Payload Index

Qdrant also indexed `release_year` and `genre_ids` as integer fields, and `industry` and `country_codes` as keyword fields. These fields matched the benchmark's structured filters and supported restricting eligible points within the vector store during filter-then-search.

Index definitions were consistent across layouts and were neither tuned nor evaluated as a separate design variable. They describe the configured access paths, not evidence that every request used a particular index.

3.3 Software Architecture

The orchestrator coordinates requests over the aligned data. This section describes its components, relational and vector execution paths, request validation, timing and deployment.

3.3.1 Components and Technology Choices

A single orchestration service coordinates two PostgreSQL-Qdrant shard pairs. Two shards are used to exercise broadcast and selective querying, while keeping the experiment small enough to inspect individual results. Figure - 4 shows this deployment topology.

Rust suited the service's need to coordinate concurrent work with explicit request and error types. Axum supplied the HTTP boundary, while Tokio supported asynchronous calls to independent shards. Concurrent broadcast execution matters to the comparison: serialising otherwise independent shard calls would add their waiting times and change the baseline against which selective execution was assessed. This is a suitability argument for the chosen stack, not evidence that Rust or these libraries outperform alternatives, since no language or framework comparison was conducted.

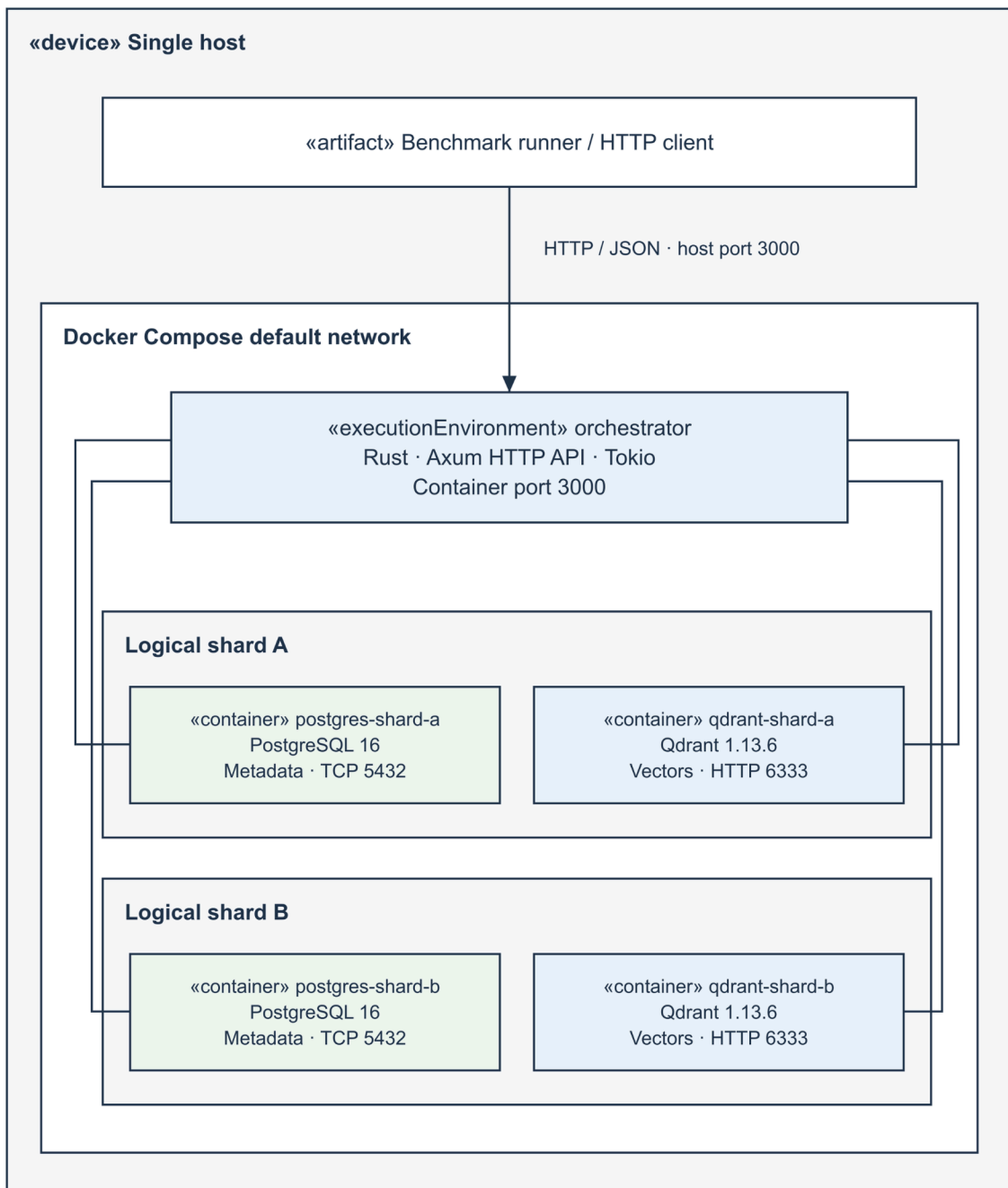
PostgreSQL stored the authoritative movie metadata and relationships. Its relational model was appropriate for exact conditions over dates, genres, people, industries, and countries. Diesel and diesel-async provided database access, schema integration, and reusable asynchronous connection pools. Qdrant stored the dense and sparse representations and the payload fields needed for filtering. Its support for named vectors and separate collections allowed the retrieval representations and data layouts to coexist.

Docker Compose described the service topology, network settings, and persistent database volumes. The configured database images were PostgreSQL 16 and Qdrant 1.13.6. The deployment recorded image tags, but some tags could resolve to updated images over time. Consequently, image tags alone do not establish an exact record of every installed component.

Figure - 5 separates responsibilities that require different checks: request decoding, validation and coordination, store access, and enrichment. Keeping these boundaries within one service allowed them to be tested separately without adding inter-service calls to the measured retrieval path. The HTTP boundary decodes the request type,

System deployment

UML deployment view · five containers on one host

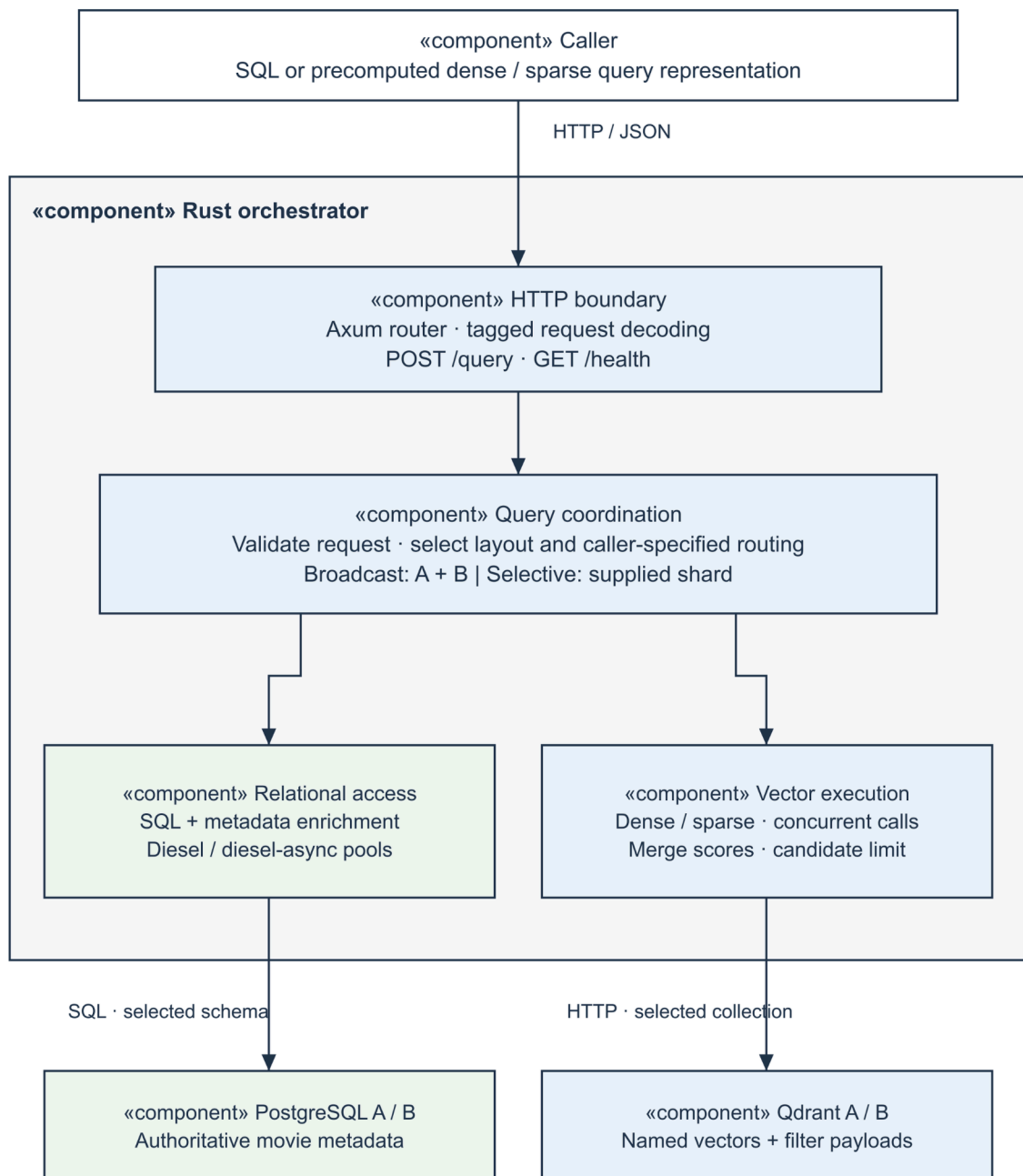


Each store has its own persistent volume. Hash and industry layouts coexist in the same containers. Shard pairing means aligned movie IDs; it does not denote a direct database-to-database connection.

Figure 6: Deployment

Orchestrator components

UML component view · logical responsibilities in one Rust service



Coordination preserves vector rank during enrichment, applies post-search filters, and records timings. Arrows show calls; returns are omitted. GET /health returns OK without probing the stores.

Figure 7: Orchestrator

coordination selects the execution path, and relational access supplies authoritative metadata for vector results. Separate deployments might permit independent scaling, but that was not required for the sequential two-shard experiment.

3.3.2 Relational Orchestration

Broadcast querying provided a baseline that did not require prior knowledge of which shard contained matching records. Selective querying tested whether avoiding a shard was beneficial when its exclusion was justified. Supplying the layout, routing strategy, SQL statement, result limit and any selected shard explicitly made each execution reproducible and separated routing execution from the unresolved problem of inferring the route. Broadcast statements ran concurrently on both shards; selective statements ran only on the supplied shard.

The orchestrator appended the rows from shard B to those from shard A and applied the requested global row limit. It did not perform a global sort, remove duplicates, or reduce aggregate results. Ordering within each shard was determined by the SQL statement. For the benchmark's count queries, the evaluator added the per-shard count values before comparing the answer with the ground truth.

SQL execution took place within a read-only transaction. A transaction-local search path selected the appropriate layout and was discarded when the transaction ended. This prevented a reused connection from retaining the previous request's layout setting. A reusable pool of up to four connections was maintained for each PostgreSQL instance.

Selective routing validated the presence of a shard identifier but did not establish whether that shard contained every matching record. The supplied configuration therefore remained part of the request's correctness assumptions. Similarly, the prototype accepted supplied SQL and did not implement a general distributed query planner.

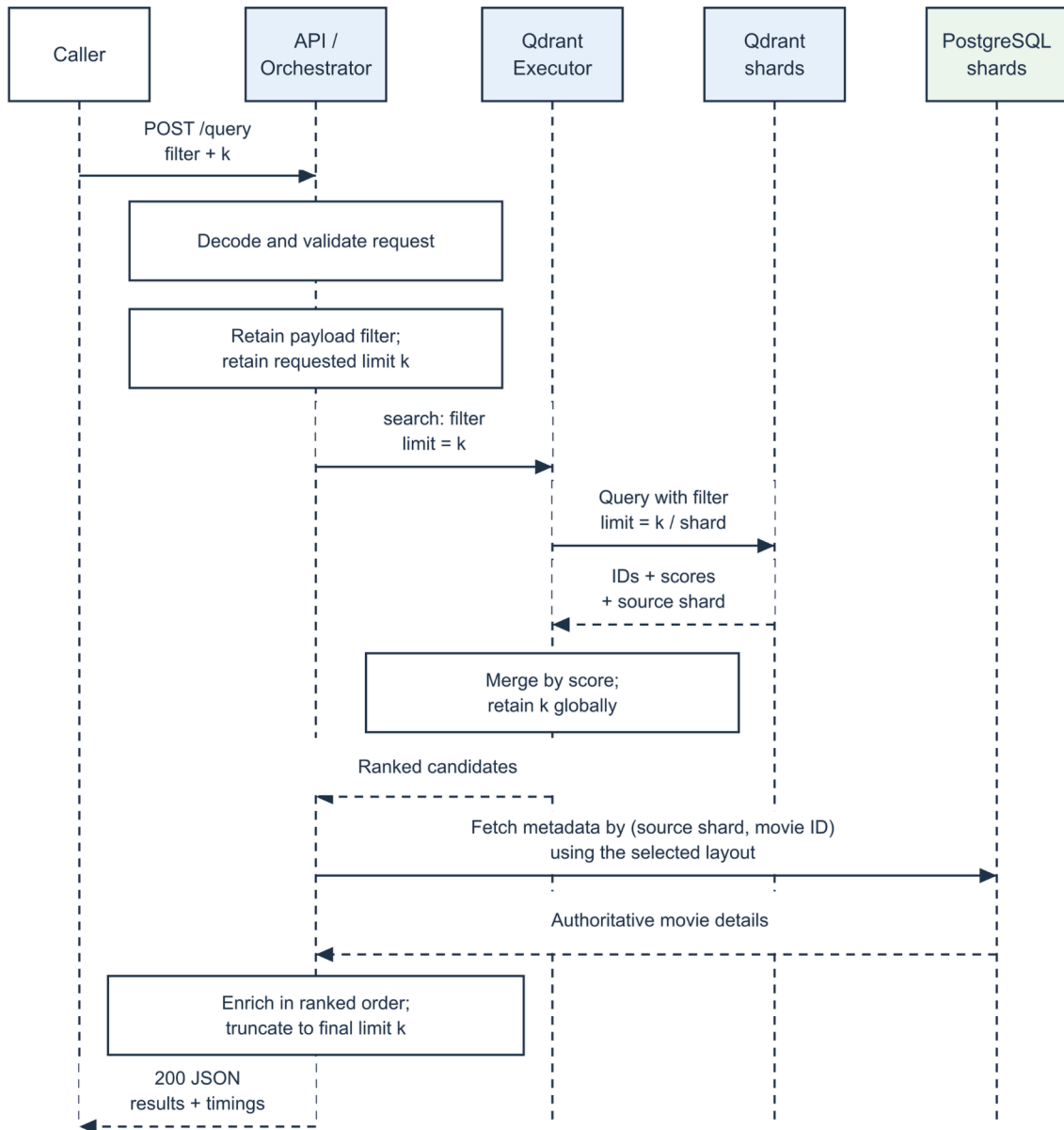
3.3.3 Vector Retrieval and Enrichment

Dense and sparse retrieval shared an API and execution path so that orchestration differences would not be confounded by separate service implementations. Tagged request types preserved their distinct input requirements and rejected incompatible fields. Dense requests supplied a supported vector name and a 1,024-value representation, while sparse requests supplied indices and weights.

Broadcast vector querying searched both Qdrant shards concurrently. The

Filter-then-search

UML sequence view · successful filtered dense or sparse request

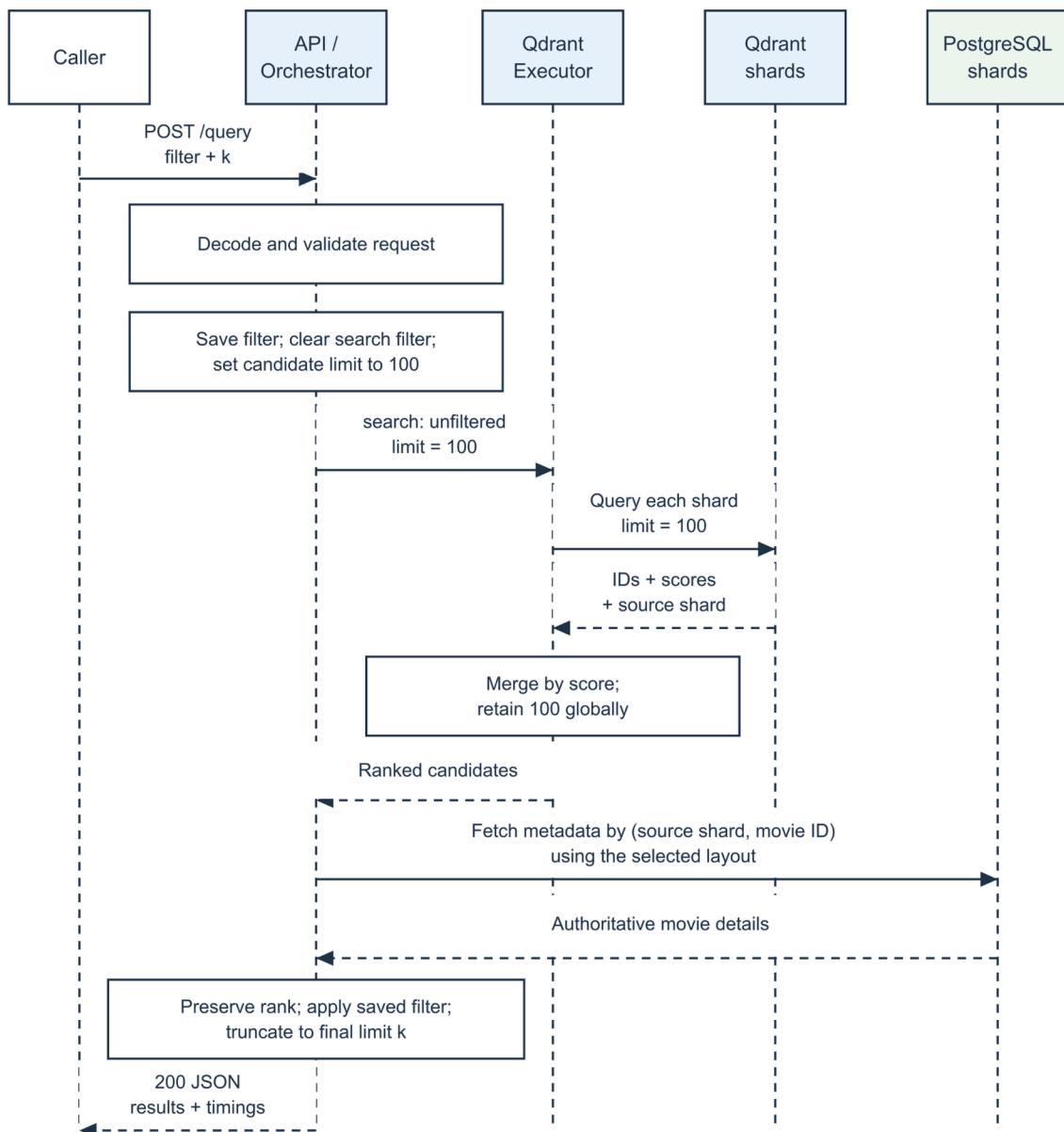


Shard lifelines abbreviate A and B. Broadcast calls both concurrently; selective calls the caller-specified shard. Metadata loads run concurrently for source shards with candidate IDs; empty ID lists cause no SQL query. Qdrant filters; PostgreSQL enriches, without a pre-search ID set. Failure paths omitted: a required store failure ends the request in error.

Figure 8: Filter-then-search

Search-then-filter

UML sequence view · successful filtered dense or sparse request



Shard lifelines abbreviate A and B. Broadcast calls both concurrently; selective calls the caller-specified shard. Metadata loads run concurrently for source shards with candidate IDs; empty ID lists cause no SQL query. Filtering can yield fewer than k results; the pool is not refilled. Failure paths omitted: a required store failure ends the request in error.

Figure 9: Search-then-filter

orchestrator combined the candidates, sorted them by descending score, and applied a global limit. Selective querying contacted one specified shard. Each candidate retained its source shard so that the corresponding PostgreSQL instance could supply the authoritative movie details.

Enrichment used the shared `movie_id` described in Section 3.3.2 to obtain authoritative details from the corresponding PostgreSQL shard. Grouping identifiers by source shard allowed lookups to proceed concurrently, while reconstructing the vector order prevented database row order from changing the ranking being evaluated. Returning only Qdrant payloads would avoid this lookup but would not provide the full relational metadata. A missing relational record caused failure because silently omitting it would conceal broken alignment and alter the returned result set.

For filtered vector requests, filter-then-search passed the structured conditions to Qdrant during retrieval. Search-then-filter instead requested up to 100 unfiltered candidates from each contacted shard. The executor merged these candidates and retained 100 globally before enrichment. The orchestrator then applied the conditions to the authoritative metadata and imposed the final result limit.

The two filter orders expose a specific trade-off. Early filtering restricts the search to eligible candidates; later filtering permits selection against authoritative relational metadata but can discard candidates after the search budget is exhausted. A fixed global pool of 100 bounded the work of search-then-filter and made its execution repeatable. It was not an experimentally established optimum. Conditions on different fields used AND, alternative genres or countries within a field used OR, and year bounds were inclusive, giving both paths the same intended predicate semantics.

Figures 6 and 7 show the successful API request sequences for the two execution orders. In filter-then-search, Qdrant applies the payload conditions during retrieval; PostgreSQL is accessed afterwards for enrichment. In search-then-filter, the orchestrator removes the filter from the vector request, retrieves and merges the candidate pool, and applies the saved conditions after enrichment. The latter can return fewer than the requested number of results because rejected candidates are not replaced.

3.3.4 Validation, Error Handling and Timing

Validation preceded database access so that malformed requests could not become strategy-dependent database work. Invalid limits, incompatible fields, unsupported

vector settings and inconsistent routing fields were rejected before execution. Required-store failures produced errors: accepting a partial response could make a strategy appear faster while omitting part of its intended search space. This favoured explicit correctness over partial availability for the experiment.

The API distinguished invalid requests, unavailable stores, timeouts, and unreadable Qdrant responses. These distinctions supported diagnosis during development and allowed unsuccessful requests to be identified during evaluation.

Relational responses included per-shard and total orchestration timings. Vector responses included vector retrieval time, enrichment time, and total orchestration time. Enrichment time also included post-search filtering and final truncation where applicable. Per-shard durations were not added to obtain end-to-end latency because concurrent operations could overlap.

3.3.5 Startup and Deployment

At startup, the service loaded the database URLs and bind address, constructed the connection pools and HTTP client, and started the API server. A five-second timeout was configured for shard operations. This timeout applied to individual operations rather than to the complete sequence of vector retrieval and enrichment.

The query endpoint accepted the three request types. A separate health endpoint returned a static response indicating that the process could handle an HTTP request. It did not verify that every database was ready, and successful pool construction did not itself establish a database connection.

The Dockerfile compiled the Rust service in release mode using the Cargo lockfile and copied the executable into a runtime image. Compose configured persistent volumes for the databases. These arrangements supported repeated deployment, although exact image digests would be required to identify an immutable container environment.

3.4 Verification and Testing

Verification targeted the assumptions on which a meaningful latency comparison depended: identical inputs, aligned stores, valid routes, preserved rankings and correct metric calculations. A fast response would be uninformative if it used the wrong data or dropped required results. Python tests covered preparation, manifests, ingestion validation, query definitions, representations and benchmark calculations. Rust tests covered validation, routing, database execution, merging, enrichment and

errors.

Controlled inputs and mock execution boundaries made validation and failure cases repeatable without requiring a populated deployment. Real-store integration tests were also necessary because mocks could not establish compatibility with database schemas, filter behaviour or metadata conversion. The two levels therefore addressed different risks; successful component tests alone would not establish correctness of the complete retrieval path.

Observed defects and the corrections made during verification are discussed in Section 4.2.

3.5 Evaluation Method

The completed orchestrator was evaluated using a fixed workload to examine the routing and execution alternatives identified in the project objectives. The following subsections define the workload, measures, execution procedure, and method of comparison. The recorded findings and their interpretation are presented in Chapters 4 and 5.

3.5.1 Benchmark Queries and Ground Truth

The workload separated exact relational answers, semantic target recovery and lexical known-item retrieval because each required a different definition of success. It contained 12 queries of each type, giving each retrieval path equal logical-query coverage. Six dense and six sparse queries included structured filters so that filter order could be compared where it affected execution. The filters covered year ranges, genres, industries, countries and combinations. This was a deliberately constructed coverage set, not a sample established as representative of enterprise traffic.

Relational ground truth was represented as an expected count or list of movie identifiers. Dense queries each specified one judged relevant target. Sparse known-item queries also specified a designated movie identifier, allowing retrieval success and target rank to be measured. The default vector result limit was ten.

Precomputed query representations held the input vectors constant and excluded embedding-service latency from the orchestration comparison. Models and dimensions matched the stored movie representations. Using `overview_dense` for all dense benchmark queries likewise kept document representation fixed across strategies. This isolated the tested execution choices but left comparison with full-

document embeddings outside the evidence produced.

The queries expanded into 109 configurations: 25 relational, 36 dense, and 48 sparse. Each configuration specified the layout, routing strategy, and execution order where applicable. The workload did not include every possible combination. Selective configurations required a justified shard selection, and comparisons between filter orders used filtered queries.

For some sparse known-item queries, the target identifier determined its shard. These configurations measured retrieval with the correct shard supplied; they did not evaluate automatic inference of shard location from the query text. Furthermore, the single-target dense judgements did not establish the relevance of every other movie in the corpus.

3.5.2 Correctness, Latency and Retrieval Effectiveness

Relational correctness was checked by summing per-shard count values or comparing sorted identifier lists with the ground truth. Latency differences were considered alongside correctness because an incomplete answer would not constitute a successful optimisation.

Client-observed latency measured the complete retrieval exchange seen by the caller, while server timings helped locate costs within orchestration. Both were needed because a client difference could arise outside shard execution. Mean latency described average cost; P95 and P99 described the slower observations that the mean could conceal. Percentiles used the nearest-rank definition. With 30 observations per configuration, P99 was the maximum, so including it fulfilled the planned metric coverage without establishing a reliable estimate of rare delays.

Dense retrieval effectiveness was measured using precision, recall, F1-score, Hit Rate@k, and binary NDCG@k. Precision@k divided the number of judged relevant returned movies by k, including when fewer than k results were returned. Recall@k divided the same count by the size of the relevance set. F1-score combined precision and recall through their harmonic mean, with zero used when both values were zero.

Hit Rate@k indicated whether a designated relevant item appeared among the first k results. NDCG@k additionally accounted for its position by discounting relevance at rank r by $1/\log_2(r + 1)$ and normalising against the ideal ranking. Consequently, it distinguished an early relevant result from a later one. As the judgements were

binary, it did not distinguish degrees of relevance.

Precision and recall address different concerns when relevance sets contain several items: the concentration of relevant results and the proportion recovered. F1 combines them, while NDCG distinguishes rankings with the same recovered items. In this benchmark, however, one dense target and $k = 10$ meant Precision@10 could not exceed 0.1 and Recall@10 equalled the hit indicator. Precision, recall and F1 were retained to report the planned measures, but offered limited independent information under these judgements. Other returned movies could be relevant despite not being judged.

Sparse known-item retrieval used Hit Rate@k and mean reciprocal rank (MRR). Reciprocal rank was $1/r$ when the target was retrieved and zero otherwise. MRR distinguished targets placed first from those appearing lower in the ranking. Failed requests were counted separately and excluded from successful latency and retrieval averages.

3.5.3 Benchmark Execution

The recorded benchmark used Windows 11 and CPython 3.12.14, with the orchestrator and knowledge stores running through Docker Compose. The run manifest recorded checksums for the query definition and saved representations, allowing the measured workload to be identified.

One unmeasured warm-up preceded 30 measured requests per configuration. Excluding the warm-up focused the comparison on repeated retrieval rather than first-use setup; repetition allowed timing variability to be described instead of relying on one request. Neither one warm-up nor 30 repetitions was validated as sufficient for steady-state or rare-tail estimation. Sequential execution produced 3,270 measured requests and avoided overlapping client requests, but fixed configuration order left cache history and host activity as possible confounders.

Each outcome was saved immediately with its latency, server timings, ranking, scores, status, and failure details. Client timing began after request encoding and included the HTTP round trip and response decoding. A 15-second client timeout limited waiting for a response. The contacted-shards field was derived from the requested configuration, while source shards were obtained from returned results.

The run measured sequential client requests rather than throughput under concurrent load. Conditions were not randomised or interleaved, so cache history and changes in host activity could influence the observed timings.

3.5.4 Strategy Comparisons and Report Artifacts

Matched logical queries were necessary because the valid strategy groups contained different workloads. Comparing their unadjusted means could attribute an easier set of queries to a better strategy. Repetitions were first averaged within each query and configuration, then differences were calculated for queries shared by both strategies. This treated repetition as measurement of the same case rather than additional independent workload coverage.

Differences were expressed as strategy B minus strategy A, so negative latency differences favoured B. Two-sided 95% percentile-bootstrap intervals resampled the matched logical-query differences, preserving the unit on which the comparison was made rather than pooling all requests. The calculation used 10,000 resamples and seed 20260915 to make the numerical procedure repeatable. No interval was reported for a single matched query because it supplied no between-query variation to resample. These settings define the analysis procedure; they do not compensate for a small or unrepresentative workload.

The intervals describe the selected workload and environment. They do not establish that the queries represent a random sample of enterprise requests. At the individual configuration level, intervals for mean latency treated repeated timings as independent observations, an assumption that may be affected by sequential execution.

Tables and figures were generated offline from the saved request records. Separate functions prepared configuration summaries, aggregate strategy tables, matched comparisons, and chart data. This allowed report artifacts to be regenerated without issuing further retrieval requests or changing the raw measurements.

3.6 Reproducibility

Reproducibility was supported through the source code, dependency records, cached responses, canonical data, layout manifests, stored embeddings, and benchmark artifacts. Checksums linked derived files to their inputs, while saved representations avoided repeating embedding requests during evaluation.

The processing stages were deliberately separated. Data collection could be repeated from cached responses, ingestion followed the shared manifests, and benchmark analysis operated on completed request records. This made the origin of the data used by each stage traceable.

These controls supported reconstruction of the implemented system but did not remove every source of variation. Mutable container tags, host configuration, and database defaults could affect a later deployment. The recorded run environment and unresolved configuration details must therefore accompany the reported results.

3.7 Data Use and Credentials

The project used movie metadata for an academic prototype and did not collect user conversations or private enterprise documents. External API credentials were supplied through environment variables. The local deployment also contained development database credentials in its Compose configuration and was not designed as a production access-control system.

The use and presentation of the source data remain subject to TMDB's terms and attribution requirements.

4. Results and Analysis

This chapter presents the recorded benchmark outcomes and examines their implications for routing and execution order. All numerical results use the artifact set run-2026-09-16; the earlier run and diagnostic requests are not pooled with it. Comparisons use the methods defined in Section 3.6, with matched logical queries as the basis for claims about strategy differences.

4.1 Benchmark Outcomes

The benchmark completed 3,270 measured requests across 109 configurations, with 30 repetitions per configuration and no recorded request failures. All 750 relational checks matched their expected results. Table 1 summarises the workload and effectiveness measures.

The dense averages were Precision@10 = 0.089, Recall@10 = 0.889 and F1@10 = 0.162. With one judged relevant target per query, a successful request has

Retrieval type	Logical queries	Configurations	Measured requests	Recorded outcome
Relational	12	25	750	750 of 750 correctness checks passed
Dense semantic	12	36	1,080	Hit Rate@10 = 0.889; binary NDCG@10 = 0.827
Sparse known-item	12	48	1,440	Hit Rate@10 = 1.000; MRR@10 = 0.990

Overall outcomes. Effectiveness averages include the applicable configurations and repetitions, so queries with more configurations contribute more observations. They are descriptive totals rather than equal-weight comparisons of strategies.

Table 1: Overall outcomes

Precision@10 = 0.1 even when its remaining results are useful. These values therefore measure recovery of the designated targets, not the proportion of semantically relevant movies established by exhaustive assessment.

Successful execution and successful retrieval must also be distinguished. Every dense request completed, but some did not return its designated target within the first ten results. Conversely, the absence of failures in this run establishes successful execution under the measured conditions, without establishing behaviour during outages or sustained load.

Figure 8 provides an overview of server orchestration time across all configurations. Each row identifies a query, layout, routing strategy and execution order; None means that filter order is not applicable to a relational request. Bar length shows the

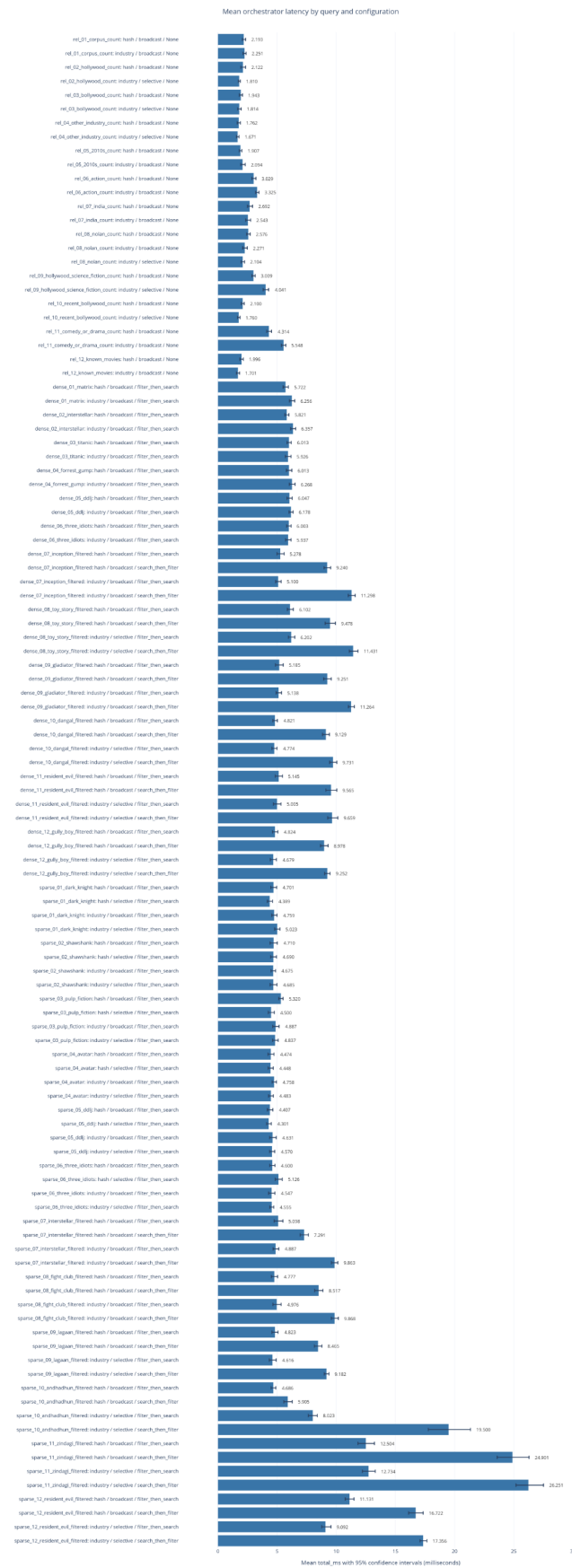


Figure 10: Mean orchestrator latency by query and configuration

mean total_ms, and the error bars show 95% bootstrap confidence intervals for that mean over 30 repetitions. These intervals describe uncertainty in the mean, not the range containing 95% of individual request times.

The chart shows why a single overall average would conceal useful variation. For the dense Inception query under hash-broadcast execution, mean orchestration time increased from 5.278 ms with filter-then-search to 9.240 ms with search-then-filter. The sparse Zindagi query under industry-selective execution increased from 12.734 to 26.251 ms. These examples show that the size of the difference depends on the query as well as the strategy. The matched analysis in Section 4.4 assesses whether the pattern extends across shared queries; visual overlap between individual configuration intervals is not a substitute for that comparison

4.2 Verification Findings

Verification exposed two ways in which nominally equivalent strategies could acquire different semantics. Search-then-filter initially transformed requests before validating them, allowing invalid caller settings to bypass checks. Validating the original request removed that inconsistency. Separately, PostgreSQL's industry label other differed from the canonical and Qdrant label other_or_ambiguous, causing post-search filtering to reject records that pre-filtering accepted. Restoring the canonical label during enrichment made the comparison concern execution order rather than inconsistent metadata interpretation.

Routing validity also required a data-level check rather than an assumption about the layout's name. The India-count smoke test exposed an incomplete selective route. Diagnostic measurements were excluded from the recorded results because their role was to verify the evaluation procedure, not estimate strategy performance.

The India count provides a concrete example of the difference between a plausible routing rule and a complete answer. Of 1,150 movies matching the production-country condition, only 1,086 were on industry shard B; the remaining 64 were Hollywood records on shard A. A country predicate therefore could not safely be treated as an industry predicate. Removing this selective configuration preserved the full-corpus ground truth rather than adjusting the expected answer to match an incomplete route.

These findings show why verification extended beyond checking HTTP success. The benchmark and regression cases exposed errors in validation order, metadata consistency and routing assumptions. The final run's correctness results apply to the

supported benchmark cases; they do not establish correctness for arbitrary SQL, every filter combination or every possible shard choice.

4.3 Relational Retrieval and Shard Routing

All relational configurations produced correct counts or identifier lists under the evaluator's comparison rules. Broadcast count responses contained a value from each shard, which the evaluator summed. This success should not be interpreted as evidence that the API performs arbitrary distributed aggregation: the service returns shard results and does not implement a general global reduction or ordering stage.

Table 2 compares relational latency on shared logical queries. Differences are B minus A; negative values favour B. Confidence intervals use the matched-query bootstrap described in Section 3.6.4.

Strategy A	Strategy B	Matched queries	Client mean difference, ms	95% confidence interval, ms
Hash, broadcast	Industry, broadcast	7	-0.014	[-2.232, 2.375]
Hash, broadcast	Industry, selective	6	-0.303	[-1.186, 0.637]
Industry, broadcast	Industry, selective	1	-1.868	Not estimated: one query

Matched relational comparisons. The second row changes both layout and routing, whereas the third holds layout constant.

Table 2: Matched relational comparisons

Neither of the multi-query intervals excludes zero. The measurements therefore do not provide clear evidence of a consistent relational latency improvement from changing layout or from the combined change to industry-selective execution. The one direct comparison within the industry layout favoured selective execution, but one query cannot establish a workload-wide effect.

Selective execution reduces the requested shard count from two to one, but this does not imply a proportional reduction in elapsed time. Broadcast requests execute concurrently, and client latency includes work outside the database operations. The result is consistent with a small local deployment in which avoiding one shard need not dominate the complete request cost. CPU utilisation, transferred bytes and throughput were not measured, so possible resource savings remain unquantified.

4.4 Filter Order and Vector Latency

For filtered vector queries, execution order gave the clearest latency differences. Table 3 holds layout and routing constant and compares filter-then-search (FTS) with

search-then-filter (STF). Each mean is calculated over the same queries in that row.

Retrieval	Layout / routing	Queries	FTS client mean, ms	STF client mean, ms	STF minus FTS, ms	95% confidence interval, ms
Dense	Hash / broadcast	6	19.581	25.378	5.798	[4.107, 7.591]
Dense	Industry / broadcast	2	22.065	25.697	3.632	[2.662, 4.602]
Dense	Industry / selective	4	22.180	25.005	2.825	[2.410, 3.336]
Sparse	Hash / broadcast	6	21.252	26.330	5.078	[1.426, 9.158]
Sparse	Industry / broadcast	2	23.092	24.718	1.626	[-0.611, 3.862]
Sparse	Industry / selective	4	22.618	33.148	10.530	[7.192, 13.868]

Matched execution-order comparisons. Intervals are pointwise, use logical queries rather than repeated requests as samples, and should be interpreted cautiously for groups of only two queries.

Table 3: Matched execution order comparisons

STF had a higher client mean in every row. Five of the six intervals excluded zero; the sparse industry-broadcast comparison remained inconclusive at the client boundary. On hash-broadcast queries, FTS reduced the client mean relative to STF by approximately 22.8% for dense retrieval and 19.3% for sparse retrieval. These reductions describe the selected filtered workload, not all vector requests.

The server timings help explain the observed cost. For dense hash-broadcast queries, STF added 4.047 ms of total orchestration time, of which 2.945 ms was additional enrichment time. For the corresponding sparse comparison, the increases were 4.797 ms and 4.634 ms. Under sparse industry-selective execution, additional enrichment time was 9.233 ms of a 9.456 ms total increase. Here, enrichment includes the subsequent filtering and truncation steps.

Figure 9 presents the server-side counterparts of the client comparisons, together with the other available strategy pairs. Rows that hold layout and routing constant while changing FTS to STF lie to the right of zero, supporting the interpretation that later filtering adds work inside the service. Several routing comparisons lie close to zero with intervals crossing it, providing less decisive evidence of a benefit. The server interval for sparse industry-broadcast STF minus FTS excludes zero even though the corresponding client interval in Table 4.3 does not; the two measurements cover different parts of the request. Rows changing both layout and routing, or both layout and filter order, describe combined changes and cannot isolate either factor.

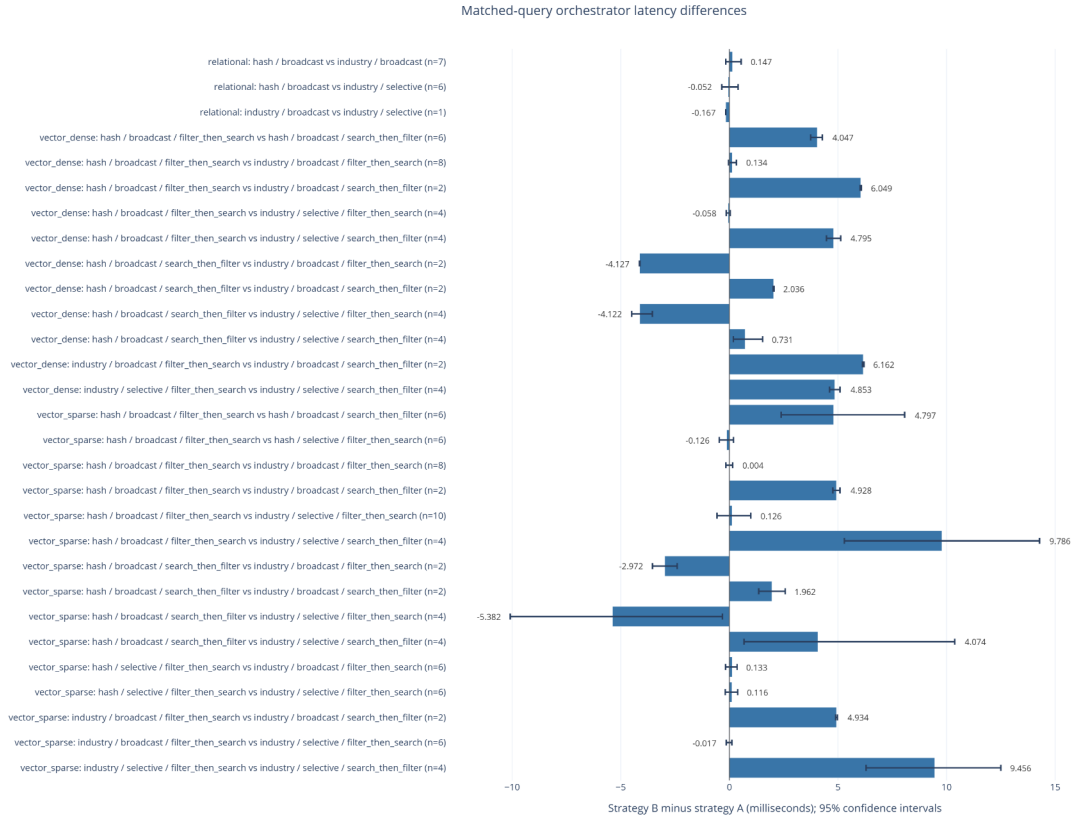


Figure 11: Matched-query orchestrator latency differences

This pattern is consistent with the implementation: FTS retains up to ten candidates before enrichment, whereas STF retains up to 100 globally and enriches them before applying the structured conditions. The measurements support the interpretation that processing the larger intermediate set accounts for much of the additional cost. They do not isolate the cost of each lookup, serialisation operation or filter predicate.

The fixed pool of 100 is part of this comparison. Changing it could change both cost and target recovery, and its optimum was not investigated. The results support early filtering for the tested implementation and workload without establishing that every form of post-filtered retrieval is slower.

4.5 Retrieval Effectiveness and Result Completeness

All six filtered dense queries retrieved their designated target at rank one under both execution orders in every applicable configuration. All six filtered sparse queries also placed their designated targets first. Thus, the additional STF latency was not accompanied by an improvement in the measured target-based quality of these matched cases.

The overall dense differences between strategy groups require a different interpretation. Hash-broadcast FTS had Hit Rate@10 = 0.833, whereas hash-

broadcast STF had 1.000. The first group contains all 12 dense queries and the second only the six filtered queries. Comparing those values as an execution-order improvement would confuse a change in workload with a change in strategy.

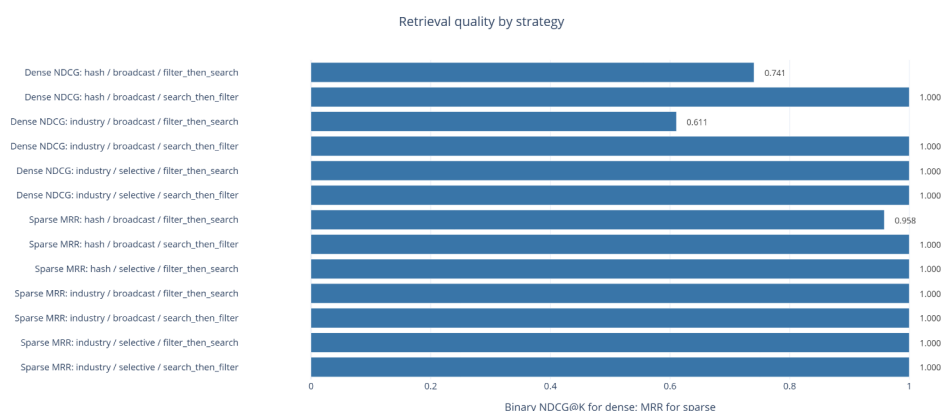


Figure 12: Strategy-group retrieval quality

Figure 10 shows dense NDCG of 0.741 for hash-broadcast FTS and 0.611 for industry-broadcast FTS, while the remaining dense groups reach 1.000. These are descriptive group averages: the lower groups include unfiltered queries whose targets were missed or ranked below first, whereas the groups scoring 1.000 contain filtered queries. The chart therefore does not establish that STF improves dense ranking. Sparse MRR is 0.958 for hash-broadcast FTS and 1.000 for the other groups, indicating a lower target position in at least one case despite successful recovery. Comparing the lengths of dense and sparse bars would not establish that one retrieval type is better, because their queries and scoring measures differ.

The unfiltered dense queries explain the lower overall effectiveness. The Matrix and Dilwale Dulhania Le Jayenge targets were absent from the top ten in both broadcast layouts. Titanic appeared at rank five, yielding binary NDCG@10 of approximately 0.387, while 3 Idiots appeared at rank three, yielding 0.500. Interstellar and Forrest Gump appeared first. These target outcomes were consistent across the two layouts in the recorded requests. They identify cases for further relevance assessment, but cannot establish whether the other retrieved movies were unsuitable because those movies were not exhaustively judged.

Sparse retrieval recovered every designated target in all configurations. The Avatar query placed its target second under hash-broadcast execution and first in its other configurations; this accounts for the overall MRR below one. Sparse scores were merged across collections without global normalisation, and the collections used their own IDF statistics. This makes score comparability a possible contributor to ranking

differences, but the benchmark contains no controlled score-normalisation experiment that would establish the cause.

Target success also conceals differences in the number of results returned. Table 4 illustrates this using counts verified in the raw rankings across repetitions.

Query	Layout / routing	FTS results	STF results	Designated target
Dense: Dangal	Hash / broadcast	10	6	First in both
Dense: Gully Boy	Hash / broadcast	9	4	First in both
Dense: Gully Boy	Industry / selective	9	7	First in both
Sparse: Interstellar	Hash / broadcast	10	4	First in both
Sparse: Resident Evil	Hash / broadcast	10	5	First in both

Examples of reduced result counts under STF at a requested limit of ten. Counts describe returned results rather than additional relevance judgements.

Table 4: Result counts by execution order

STF cannot replace candidates discarded after its global pool has been truncated. Consequently, relevant or eligible movies outside that pool cannot be returned even if fewer than ten candidates survive. These cases demonstrate a practical limitation that Hit Rate@10 alone misses. Returning more movies does not itself establish greater semantic relevance, but an application requesting a set of options may care about this difference even when its designated target remains first.

4.6 Layout, Routing and Timing Boundaries

The sparse unfiltered known-item queries allow direct routing comparisons within each layout. Across six matched queries, selective minus broadcast client latency was -0.651 ms for the hash layout, with a 95% interval of [-3.222, 1.410] ms. For the industry layout it was -1.175 ms, with an interval of [-3.654, 1.906] ms. Neither interval excludes zero. These configurations also supplied the correct target shard, so they measure execution after successful resource selection rather than the cost or accuracy of discovering that shard.

Some differences appeared at the client boundary without corresponding evidence of a server-side change. Across eight matched sparse queries using broadcast FTS, industry minus hash client latency was 3.680 ms [1.877, 5.649], while the

orchestration difference was 0.004 ms [-0.160, 0.151]. For four filtered dense queries, changing from hash-broadcast FTS to industry-selective FTS increased the client mean by 3.797 ms [1.884, 5.711], while the server difference was -0.058 ms [-0.142, 0.040]. The latter comparison changes two factors at once.

These discrepancies prevent a simple attribution of client differences to shard-search work. Client timing includes the HTTP exchange and response decoding, while server timing covers the internal orchestration path. Sequential configuration execution also permits time-varying host activity to affect the comparisons. The available timing fields do not identify which of these factors caused the discrepancy, and subtracting the timers does not isolate network latency.

Mean latency must be read alongside tail behaviour. For example, the pooled sparse industry-selective STF group had client mean, P95 and P99 of 33.148, 50.307 and 54.559 ms respectively. These are pooled percentiles over 120 requests from four queries; they are not averages of configuration percentiles. For individual configurations, 30 repetitions make nearest-rank P99 the maximum observation. These measures describe the recorded variability but do not provide a robust estimate of rare production delays.

5. Evaluation

The project aimed to implement a deterministic orchestrator and establish how routing and execution strategies affect retrieval latency and effectiveness. This chapter assesses the extent to which that aim was achieved, the validity of the evidence and the improvements needed before broader conclusions can be drawn.

5.1 Achievement of the Objectives

Table 5 evaluates the five objectives stated in Section 1.3 of the existing report. Implementation success is distinguished from the strength of the experimental evidence.

Objective	Assessment	Evidence and qualification
1. Implement a stateless HTTP orchestration framework for relational, dense and sparse retrieval over aligned data	Achieved within the prototype scope	Three request types share one service, with aligned identifiers and manifests, concurrent shard access and vector enrichment. The recorded run exercised all three paths.
2. Implement and compare deterministic routing and filter order	Achieved for the configured cases	Broadcast, caller-supplied selective routing and both filter orders were implemented and compared. Automatic shard selection and a fully crossed experiment were not provided.
3. Establish a reproducible benchmark with development and held-out test queries and appropriate ground truth	Partially achieved	Versioned queries, stored representations, explicit expected results, raw records and analysis artifacts support repeatability. An independently held-out test set and the full provenance of relevance assessment are not established by the available record.
4. Evaluate latency and retrieval effectiveness under controlled configurations	Achieved within the stated measurement limits	Mean, P95, P99, precision, recall, F1, binary NDCG and known-item success were recorded; MRR adds target-rank information. Single-target judgements and limited tail samples restrict interpretation.
5. Analyse conditions governing latency-quality trade-offs and explain limitations	Achieved for the selected workload	Matched comparisons identify the cost of STF, target preservation, reduced result counts and uncertain routing gains. The conclusions are bounded by the workload and single-host deployment.

Assessment against the final report's stated objectives.

Table 5: Assessment of project objectives

The main engineering contribution is the coordination of several retrieval paths under explicit execution rules, with consistent data alignment and traceable measurements. Its empirical contribution is narrower: a controlled comparison of those rules within one implementation. It does not establish the superiority of deterministic orchestration over a learned router because no such baseline was tested.

5.2 Interpretation in Relation to the Literature Review

The findings give concrete support, within this prototype, to the early-selection rationale discussed in Section 2.7. Applying structured conditions before enrichment reduced the intermediate set and usually reduced latency. This connection is supported both by the execution path and by the measured enrichment differences. It is more informative than observing that one strategy was faster without identifying the work it avoided.

The resource-selection arguments in Sections 2.3 and 2.6 receive more qualified support. Selective execution contacted fewer shards, but the recorded client latency differences did not consistently favour it. Broadcast operations ran concurrently, and the deployment contained only two local shards. Reducing the number of contacted resources therefore did not automatically translate into a measurable elapsed-time advantage. The experiment cannot determine whether savings would become greater with more shards, remote machines or concurrent clients.

Data placement also constrained which routes were correct. Industry partitioning enabled selective execution for some industry predicates but produced an imbalanced 9,559/1,534 split. Hash partitioning balanced record counts more closely but supplied little domain information for routing. The India-count defect illustrates why a partitioning rule must be considered together with predicate semantics. A deterministic rule is inspectable, but inspectability alone does not make it complete.

The results do not provide an absolute comparison with published retrieval latency or accuracy figures. Such a comparison would require compatible data, query tasks, hardware, timing boundaries and relevance judgements. The common implementation supplies an internal basis for comparing strategies, while the literature supplies the rationale for those comparisons.

5.3 Validity and Generalisability

5.3.1 Workload and Relevance Assessment

The 36 logical queries are a small, deliberately constructed workload over movies. Repeating them improves the description of timing variability; it does not increase the diversity of retrieval tasks. The dense judgements name one target per query, while sparse queries are known-item searches. Their scores cannot support a conclusion that sparse retrieval is generally more effective than dense retrieval because the tasks and representations differ.

There was a separate pilot, but the development history also records smoke testing and corrections using the final query definitions. A versioned query set is therefore insufficient evidence of a completely independent held-out evaluation. Without documented query authorship, assessment procedure, overlap and the point at which development decisions were frozen, the results should be treated as a repeatable benchmark rather than proof of generalisation to unseen queries. This is the principal shortfall against Objective 3.

The popularity-limited corpus and its two country/language discovery groups further restrict representativeness. Conclusions cannot automatically be transferred to other languages, enterprise documents, larger collections or more complex relational workloads.

5.3.2 Experimental Control and Statistical Interpretation

Configuration order was fixed rather than randomised or interleaved. One warm-up request may not have stabilised every cache, and requests within a configuration may share temporal effects. Matched logical-query comparisons reduce workload mismatch, but do not remove order effects or dependence between timings.

Bootstrap intervals quantify variability among the observed matched queries under the stated resampling procedure. They do not correct biased query selection or unmeasured changes in host activity. Intervals based on two queries have particularly limited information, and an interval containing zero is not proof of equivalent performance. The reported intervals are pointwise across several comparisons; no family-wide error control is claimed.

The workload was not a complete factorial design. Some comparisons change layout and routing together, and direct within-layout routing evidence is limited for relational queries and absent for the dense selective cases. Consequently, the benchmark cannot cleanly estimate every independent factor or interaction.

5.3.3 Deployment and Configuration Boundaries

The evaluation was limited to one host, two shards, one movie corpus, and the configured retrieval representations. The containers provided separately addressable stores but did not reproduce the network conditions or resource isolation of independent machines. The workload measured sequential requests and did not examine throughput under sustained concurrent use.

The vector result limit and search-then-filter candidate pool were fixed. Their effect was not independently varied, and the full-document dense representation was not evaluated. The single-target relevance judgements further restricted the interpretation of retrieval effectiveness.

Replication and node-failure experiments mentioned in the proposal were not included in the recorded benchmark. The implemented error handling could report failures, but this did not amount to an evaluation of resilience. These boundaries define the scope of the conclusions drawn in the following chapters.

The available manifest records the operating system, Python version and input hashes, but not the full processor, memory, storage, Docker allocation or immutable image digests for the measured run. These omissions limit exact reproduction of the latency environment. A later machine inspection cannot retroactively establish the original run's conditions.

Collection-dependent sparse IDF and approximate dense search also complicate interpretation across layouts. There was no global-IDF or exact-search baseline to separate orchestration effects from changes in retrieval behaviour inside the stores. Likewise, the choice of dense model was not evaluated against alternatives. These settings define the tested system rather than demonstrated optimal choices.

5.4 Engineering Strengths and Remaining Limitations

Shared manifests, stable identifiers and saved representations provided a traceable basis for data alignment. Separate validation, retrieval and enrichment components made regressions easier to locate. Returning an error when enrichment could not find an expected relational record also exposed misalignment instead of silently returning incomplete metadata.

However, alignment was maintained by the preparation workflow rather than a distributed transaction. PostgreSQL replacement and Qdrant collection population have different recovery properties, and a partially completed update could leave stores inconsistent. The prototype does not demonstrate continuous synchronisation or a recovery protocol for concurrent data changes.

The relational path also has a limited merge contract. It concatenates shard results and applies a limit without globally sorting or reducing arbitrary SQL results. This can favour shard A when the limit is reached and constrains the queries for which the returned result has the intended global meaning. Read-only execution limits writes, but caller-supplied SQL and a selected search path do not constitute a production access-control boundary.

Operationally, the health endpoint establishes that the process responds, rather than that every store is ready. Per-operation timeouts are not a single request-wide deadline. Error-response tests and the failure-free benchmark provide useful functional evidence, but do not substitute for fault injection, readiness checks, load testing or recovery measurement. These limitations delimit a research prototype rather than invalidate its measured strategy comparison.

5.5 Further Work

The first priority is to strengthen the benchmark's independence and relevance assessment. A new test set should be frozen after development, with documented query authorship and multiple assessed results per dense query. Recording assessor agreement and using graded judgements where justified would allow NDCG to measure more than the rank of a single target. Returning fewer than k results should also be reported alongside target success.

The next experiment should randomise or interleave configurations, repeat runs across sessions and record hardware and container settings. Adding valid broadcast counterparts to selective cases would make more routing comparisons possible without changing layout. Larger samples would improve tail-latency estimates, while concurrent load and multi-host deployments would test throughput, contention and communication effects.

A parameter study should vary filter selectivity, requested k and candidate-pool size. It should measure target recovery, result counts and enrichment cost together, and compare the two implemented dense representations. An exact dense baseline and a controlled sparse-score normalisation experiment would help distinguish orchestration effects from index and scoring effects.

Finally, automatic deterministic shard selection could be evaluated against an explicit routing oracle and, separately, a learned baseline. Fault injection and staged data replacement would address resilience and alignment during updates. These extensions follow directly from the present evidence gaps and would broaden the contribution beyond execution with caller-supplied decisions.

6. Conclusion

This project designed and implemented a deterministic query orchestrator for relational, dense semantic and sparse lexical retrieval over aligned sharded movie data. The service exposed a stateless HTTP API, coordinated two PostgreSQL-Qdrant shard pairs, supported broadcast and selective execution, and implemented two orders of filtering and vector search. A versioned benchmark and saved request records made the resulting comparisons inspectable and repeatable.

The 109 configurations produced 3,270 successful measured requests, including 750 correct relational checks. The strongest finding was that filter-then-search reduced latency for the matched filtered workload while preserving every designated target. In the hash-broadcast configuration, its client mean was approximately 22.8% lower for dense retrieval and 19.3% lower for sparse retrieval than search-then-filter. Server timings associated much of the additional post-filtering cost with enrichment of the larger candidate set. Search-then-filter also sometimes returned fewer results despite retaining the target first.

The routing findings were less decisive. Selective execution reduced the requested shard count, but matched comparisons did not establish a consistent client-latency advantage. Correctness depended on knowing that the selected shard contained all required records, and some comparisons changed both routing and data placement. These results show that an orchestration decision should be assessed through the work it performs, the completeness of its answer and the timing boundary being measured.

The contribution is limited to the tested corpus, representations and single-host environment. Single-target judgements restrict conclusions about semantic relevance, and the available evidence does not establish an independent held-out test set. Nevertheless, the implementation and analysis demonstrate how deterministic orchestration strategies can be compared within one aligned retrieval system. The next step is a broader, independently assessed workload with controlled routing comparisons, varied filtering conditions and distributed load measurements.